Code Explanation

Configuration Module for BNCPLS IF23B 10K File Generation

This is a Python configuration module for the BNCPLS IF23B 10K File Generation project. It provides functions to create a SparkSession, parse command line arguments, and get configuration parameters.

Overview of the Code

The code is divided into three main functions: `get_spark_session`, `parse_arguments`, and `get_config`. The `get_spark_session` function creates a SparkSession with specific configurations. The `parse_arguments` function parses command line arguments using the `argparse` library. The `get_config` function retrieves configuration parameters either from command line arguments or environment variables.

Step 1: Creating a SparkSession
The `get_spark_session` function creates and returns a SparkSession instance with specific configurations. It takes an optional `app_name` parameter, which defaults to "BNCPLS_IF23B_10K_File_Generation" if not provided.

```
def get_spark_session(app_name="BNCPLS_IF23B_10K_File_Generation"):
    return (SparkSession.builder
            .appName(app_name)
            .config("spark.sql.legacy.timeParserPolicy", "LEGACY")
            .config("spark.sql.files.maxPartitionBytes", "134217728"
)
            .config("spark.sql.shuffle.partitions", "200")
            .config("spark.executor.memory", "4g")
            .config("spark.driver.memory", "4g")
            .getOrCreate())
```

Step 2: Parsing Command Line Arguments
The `parse_arguments` function uses the `argparse` library to parse command line arguments. It defines several required and optional arguments, including SQL query, XSLT file paths, output directory, and file naming parameters.

```
def parse_arguments():
    parser = argparse.ArgumentParser(description='BNCPLS IF23B 10K F
ile Generation')

    parser.add_argument('--src_sql', type=str, required=True,
                        help='SQL query to extract data from source
tables')

    parser.add_argument('--xslt_file', type=str, required=True,
                        help='Path to XSLT file for Aura 14 payloads
')

    parser.add_argument('--output_dir', type=str, required=True,
                        help='Output directory for flat files')

    return parser.parse_args()
```

Step 3: Retrieving Configuration Parameters
The `get_config` function attempts to retrieve configuration parameters from command line arguments first. If this fails, it falls back to using environment variables.

```
def get_config():
    try:
        args = parse_arguments()
        config = {
            'src_sql': args.src_sql,
            'xslt_file': args.xslt_file,
            'output_dir': args.output_dir,
            # ...
        }
    except:
        config = {
            'src_sql': os.environ.get('SRC_SQL', ''),
            'xslt_file': os.environ.get('XSLT_FILE_PATH', ''),
            'output_dir': os.environ.get('OUTPUT_DIR', '/tmp/output'
),
            # ...
        }

    return config
```

Step 4: Using Environment Variables as Fallback
If parsing command line arguments fails, the `get_config` function uses environment variables to retrieve configuration parameters. It provides default values for certain parameters if they are not set.

```
config = {
    'src_sql': os.environ.get('SRC_SQL', ''),
    'xslt_file': os.environ.get('XSLT_FILE_PATH', ''),
    'output_dir': os.environ.get('OUTPUT_DIR', '/tmp/output'),
    'batch_id': os.environ.get('BATCH_ID', ''),
    'run_id': os.environ.get('RUN_ID', ''),
    'file_prefix': os.environ.get('FILE_PREFIX', 'IF23B'),
    'file_ext': os.environ.get('FILE_EXT', 'dat'),
    'max_rows_per_file': int(os.environ.get('MAX_ROWS_PER_FILE', '10
000'))
}
```